# 05 JS Variable Scopes

## JavaScript Scoping: Complete Lecture Notes & Lab Guide

### Table of Contents



---

## Prerequisites

- VS Code installed
- Live Server extension installed
- Basic JavaScript knowledge

---

## Lab Setup

### Step 1: Create Your Project Structure

1. Create a new folder called `js-scoping-lab`
2. Open the folder in VS Code
3. Create two files:
   - `index.html`
   - `script.js`

### Step 2: Set Up HTML File

In `index.html`, add this code:

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript Scoping Lab</title>
</head>
<body>
    <h1>JavaScript Scoping Lab</h1>
    <p>Open the Console (F12) to see outputs</p>

    <script src="script.js" defer></script>
</body>
</html>
```

### Step 3: Launch Live Server

1. Right-click on `index.html`
2. Select "Open with Live Server"
3. Open Developer Tools (F12)
4. Go to the Console tab

---

## Lab Exercise 1: Understanding Basic Function Scope

### Concept

Variables defined inside a function are only accessible within that function. This is called **function scope** or **local scope**.

### Code Exercise

Add this to `script.js`:

```JS
// Example 1: Basic Function Scope
function printAge(age) {
    console.log(age);
}

printAge(29);
```

**Expected Output:** `29`

**What's happening?** The `age` parameter is scoped to the `printAge` function.

### Step 4: Try Accessing Variables Outside Their Scope

Add this code:

```JS
// Example 2: Scope Error
function printAge(age) {
    console.log(age);
}

printAge(29);
console.log(age); // This will cause an error
```

**Expected Output:** `Uncaught ReferenceError: age is not defined`

**Why?** The `age` variable only exists inside `printAge`, not outside it.

**Action:** Comment out the error line before continuing:

```JS
// console.log(age); // Error: age is not defined
```

---

## Lab Exercise 2: Block Scope with Curly Braces

### Concept

Any code within `{ }` creates a new scope block. Variables defined inside are not accessible outside.

### Step 5: Create a Block Scope

Add this code:

```JS
// Example 3: Block Scope
{
    let blockVariable = "I'm inside a block";
    console.log(blockVariable);
}

// console.log(blockVariable); // Would cause error
```

**Expected Output:** `I'm inside a block`

### Step 6: Function with Multiple Variables

Add this code:

```JS
// Example 4: Multiple Variables in Function Scope
function printAge(age) {
    let isProgrammer = true;
    console.log(age);
    console.log(isProgrammer);
}

printAge(29);
// console.log(isProgrammer); // Would cause error - not accessible here
```

**Expected Output:**

```
29
true
```

**Key Point:** Both `age` and `isProgrammer` are only accessible inside `printAge`.

---

# Lab Exercise 3: Global Scope & Scope Chain

## Concept

Variables defined outside all functions/blocks are **global** and accessible everywhere. JavaScript looks for variables from inner to outer scopes.

## Step 7: Create a Global Variable

Replace your code with:

```JS
// Example 5: Global Scope
let isProgrammer = true;

function printAge(age) {
    console.log(age);
    console.log(isProgrammer); // Can access global variable
}

printAge(29);
console.log(isProgrammer); // Also accessible here
```

**Expected Output:**

```
29
true
true
```

**What's happening?** The `isProgrammer` variable is global, so it's accessible both inside and outside the function.

---

## Lab Exercise 4: Scope Chain Lookup

### Concept

When JavaScript can't find a variable in the current scope, it looks in the outer scope, then the next outer scope, until it finds it or reaches global scope.

### Step 8: Nested Scopes

Add this code:

```JS
// Example 6: Scope Chain
let isProgrammer = true;

function printAge(age) {
    let firstName = "Kyle";

    if (true) {
        console.log(age);          // Found in function scope
        console.log(isProgrammer); // Found in global scope
        console.log(firstName);    // Found in function scope
    }
}

printAge(29);
```

**Expected Output:**

```
29
true
Kyle
```

**How it works:**

1. Inside the `if` block, JavaScript looks for `age` → not found locally → looks in function scope → found!

2. Looks for `isProgrammer` → not in function scope → looks in global scope → found!
3. Looks for `firstName` → not in `if` block → looks in function scope → found!

---

# Lab Exercise 5: Variable Shadowing

## Concept

When you declare a variable with the same name in an inner scope, it "shadows" (hides) the outer variable.

## Step 9: Understanding Shadowing

Add this code:

```JS
// Example 7: Variable Shadowing
let isProgrammer = true;

function printAge(age) {
    let firstName = "Kyle";

    if (true) {
        let firstName = "Sally"; // Shadows the outer firstName
        console.log(firstName);
    }

    console.log(firstName); // Still "Kyle" here
}

printAge(29);
```

**Expected Output:**

```
Sally
Kyle
```

**Key Point:** The inner `firstName` shadows the outer one, but only inside the `if` block. Outside that block, the original `firstName` is still accessible.

---

# Lab Exercise 6: Understanding JavaScript's Scope Levels

## Concept

JavaScript has multiple scope levels: Block, Local (Function), Script/Module, and Global.

## Step 10: Add a Debugger Statement

Create a new comprehensive example:

```JS
// Example 8: Multiple Scope Levels
let isProgrammer = true; // Script scope

function printAge(age) {
    let other = "other"; // Local (function) scope

    if (true) {
        const firstName = "Sally"; // Block scope
        debugger; // Pause here to inspect scopes
        console.log(age);
        console.log(isProgrammer);
        console.log(firstName);
        console.log(other);
    }
}

printAge(29);
```

## Step 11: Inspect Scopes in DevTools

1. Save the file
2. Go to your browser
3. The page will pause at the `debugger` statement
4. Click on the "Scope" section in DevTools
5. You'll see:
   - **Block scope:** `firstName`
   - **Local scope:** `age`, `other`
   - **Script scope:** `isProgrammer`
   - **Global scope:** All browser globals + `printAge` function

**Remove the debugger line when done exploring.**

---

# Lab Exercise 7: The Difference Between let, const, and var

## Concept

- `let` and `const` are **block-scoped**
- `var` is **function-scoped** (ignores block boundaries)

## Step 12: Block Scope with let/const

Add this code:

```JS
// Example 9: let/const Block Scope
function testBlockScope() {
    if (true) {
        let blockLet = "I'm block-scoped";
        const blockConst = "Me too";
        console.log(blockLet);
        console.log(blockConst);
    }

    // console.log(blockLet);   // Error: not accessible
    // console.log(blockConst); // Error: not accessible
}

testBlockScope();
```

**Expected Output:**

```
I'm block-scoped
Me too
```

## Step 13: Function Scope with var

Add this code:

```JS
// Example 10: var Function Scope
function testVarScope() {
    if (true) {
        var varVariable = "I'm function-scoped";
    }

    console.log(varVariable); // Works! var ignores block boundaries
}

testVarScope();
```

**Expected Output:** `I'm function-scoped`

**Important:** `var` is accessible throughout the entire function, even outside the `if` block where it was declared!

---

# Lab Exercise 8: var Hoisting

## Concept

`var` declarations are "hoisted" to the top of their function scope and initialized as `undefined`.

## Step 14: Understanding Hoisting

Add this code:

```JS
// Example 11: var Hoisting
function demonstrateHoisting() {
    console.log(hoistedVar); // undefined (not an error!)

    if (true) {
        var hoistedVar = "I was hoisted";
    }

    console.log(hoistedVar); // "I was hoisted"
}

demonstrateHoisting();
```

**Expected Output:**

```
undefined
I was hoisted
```

**What's happening?** JavaScript treats the code as if it were written like this:

```JS
function demonstrateHoisting() {
    var hoistedVar; // Declaration hoisted to top
    console.log(hoistedVar); // undefined

    if (true) {
        hoistedVar = "I was hoisted"; // Assignment stays here
    }

    console.log(hoistedVar);
}
```

---

# Lab Exercise 9: Script vs Module Scope

## Concept

- **Script scope:** Variables accessible across all script files
- **Module scope:** Variables only accessible in the current module

## Step 15: Test Script Scope

Create a new file `other.js` in your project folder:

```JS
// other.js
console.log("From other.js:", isProgrammer);
```

Update `index.html` to include both scripts:

```HTML
<script src="script.js" defer></script>
<script src="other.js" defer></script>
```

In `script.js`, make sure you have:

```JS
let isProgrammer = true;
```

**Expected Output:** `From other.js: true`

**Why?** In regular script mode, top-level `let` and `const` are in script scope (accessible across script files).

## Step 16: Test Module Scope

Change `index.html` to use modules:

```HTML
<script src="script.js" type="module"></script>
<script src="other.js" type="module"></script>
```

Refresh the page.

**Expected Output:** `Uncaught ReferenceError: isProgrammer is not defined`

**Why?** In module mode, variables are private to each file unless explicitly exported/imported.

### Step 17: Proper Module Usage

Update `script.js`:

```JS
// script.js
export let isProgrammer = true;
```

Update `other.js`:

```JS
// other.js
import { isProgrammer } from './script.js';
console.log("From other.js:", isProgrammer);
```

**Expected Output:** `From other.js: true`

---

## Lab Exercise 10: Complete Scoping Example

### Step 18: Build a Comprehensive Example

Change back to regular scripts in `index.html`:

```HTML
<script src="script.js" defer></script>
```

Clear `script.js` and add this comprehensive example:

JS

```
// Complete Scoping Example

// Global variable
const globalName = "Global User";

// Script-level variable
let scriptLevel = "Accessible in this script";

function outerFunction() {
    // Function scope variable
    const outerVar = "Outer Function";

    console.log("=== Inside outerFunction ===");
    console.log("Global:", globalName);        // ✓ Accessible
    console.log("Script:", scriptLevel);       // ✓ Accessible
    console.log("Outer:", outerVar);           // ✓ Accessible

    function innerFunction() {
        // Inner function scope
        const innerVar = "Inner Function";

        console.log("\n=== Inside innerFunction ===");
        console.log("Global:", globalName);    // ✓ Accessible
        console.log("Script:", scriptLevel);   // ✓ Accessible
        console.log("Outer:", outerVar);       // ✓ Accessible
        console.log("Inner:", innerVar);       // ✓ Accessible

        if (true) {
            // Block scope
            const blockVar = "Block Scope";

            console.log("\n=== Inside if block ===");
            console.log("Global:", globalName);    // ✓ Accessible
            console.log("Script:", scriptLevel);   // ✓ Accessible
            console.log("Outer:", outerVar);       // ✓ Accessible
            console.log("Inner:", innerVar);       // ✓ Accessible
            console.log("Block:", blockVar);       // ✓ Accessible
        }

        // console.log(blockVar); // ✗ Error - block scope
    }

    innerFunction();
    // console.log(innerVar); // ✗ Error - inner function scope
}
```

```
outerFunction();
// console.log(outerVar); // ✗ Error - function scope
```

**This demonstrates the complete scope chain from innermost to outermost!**

---

## Summary: The Golden Rules of JavaScript Scoping

1. **Variables are accessible inside their scope and all inner (nested) scopes**
2. **Variables are NOT accessible outside their scope**
3. **JavaScript searches for variables from inner to outer scopes** (scope chain)
4. **let and const are block-scoped** (respect `{ }`)
5. **var is function-scoped** (ignores blocks, only respects functions)
6. **Global variables are accessible everywhere** (but use sparingly!)
7. **Inner variables can shadow outer variables** with the same name
8. **Module scope keeps variables private** unless exported

---

## Common Scoping Mistakes to Avoid

```JS
// ❌ Mistake 1: Trying to access block-scoped variable outside
if (true) {
    let x = 5;
}
console.log(x); // Error!

// ✅ Solution: Declare in outer scope
let x;
if (true) {
    x = 5;
}
console.log(x); // Works!

// ❌ Mistake 2: Using var in loops
for (var i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 100);
}
// Logs: 3, 3, 3 (not 0, 1, 2!)

// ✅ Solution: Use let
for (let i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 100);
}
// Logs: 0, 1, 2
```

---

## Quick Reference Table

| Keyword | Scope Type | Hoisted? | Redeclarable? | Best Use |
|---|---|---|---|---|
| `const` | Block | No | No | Default choice - use for constants |
| `let` | Block | No | No | When you need to reassign values |
| `var` | Function | Yes | Yes | Avoid - legacy code only |

---

# Practice Challenges

## Challenge 1: Debug the Scope Error

```JS
function challenge1() {
    if (true) {
        const secret = "hidden";
    }
    return secret; // Fix this!
}
```

**Solution:** Move `const secret` outside the `if` block or return inside the `if`.

## Challenge 2: Predict the Output

```JS
let a = 1;
function test() {
    let a = 2;
    if (true) {
        let a = 3;
        console.log(a);
    }
    console.log(a);
}
test();
console.log(a);
```

**Answer:** `3, 2, 1` (variable shadowing at each level)

---

# Conclusion

Understanding scoping is fundamental to writing bug-free JavaScript. Remember:

- **Think in layers** - scopes are nested like Russian dolls
- **Use let/const** - they're more predictable than var
- **When in doubt, use the debugger** - inspect the Scope panel in DevTools
- **Keep scope tight** - declare variables in the smallest scope needed

Keep practicing these concepts, and scoping will become second nature!